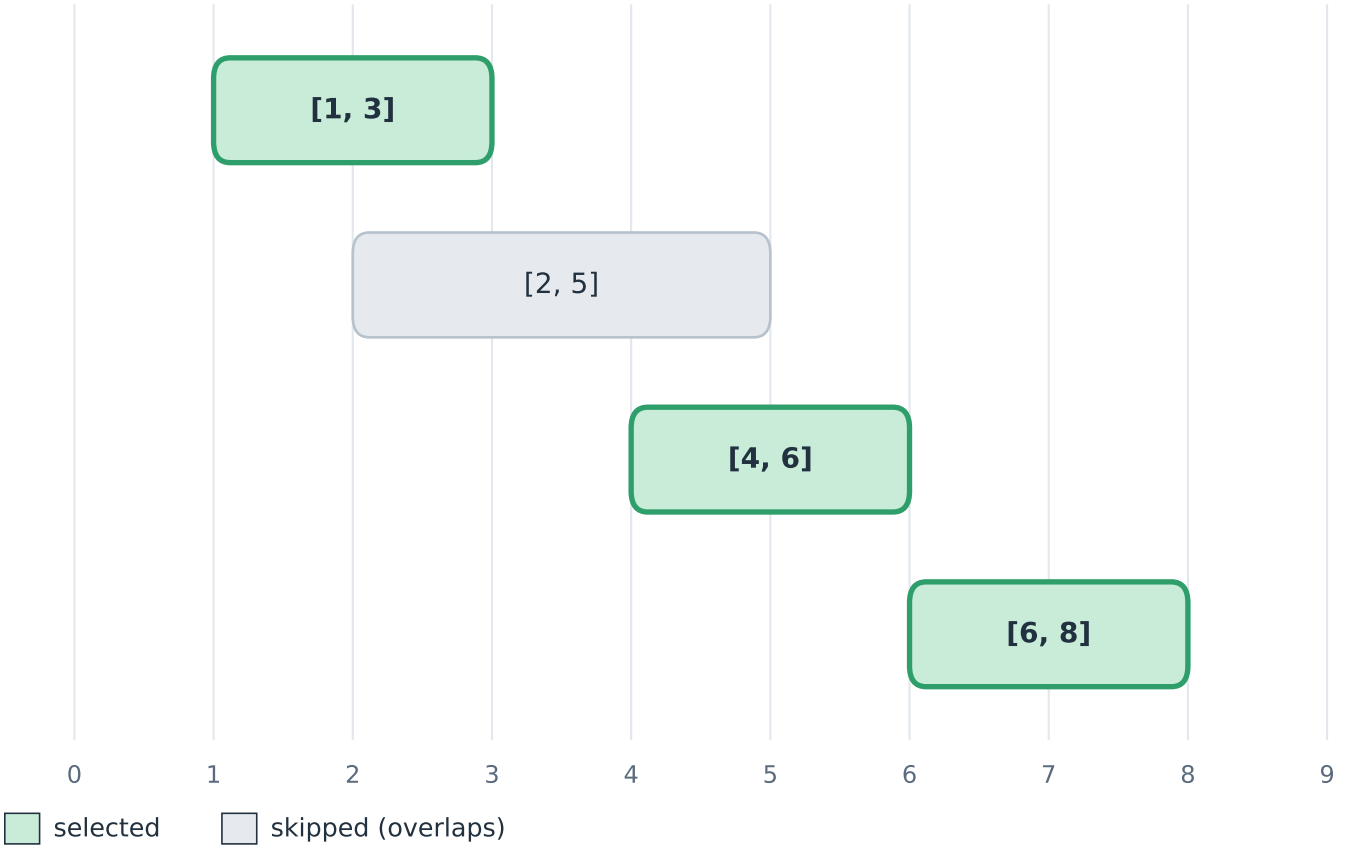
M20-01 — Activity Selection

Difficulty: Easy | Category: Greedy Algorithms



Sample input

```
4
1 3
2 5
4 6
6 8
```

Sample output

```
3
```

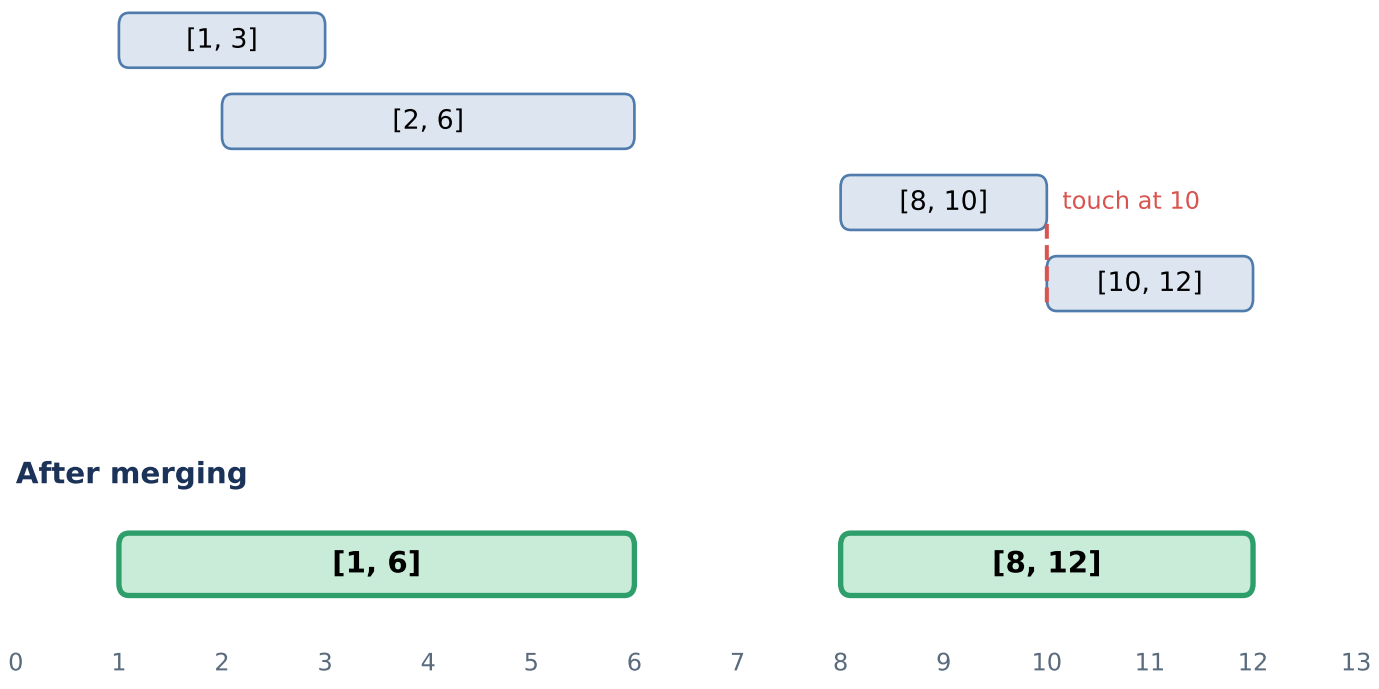
Bars show each activity's time span. Green bars are the activities chosen by earliest finish time.

Walkthrough

Sorted by finish time, take [1,3]; [2,5] overlaps it and is skipped; take [4,6]; [6,8] starts exactly when [4,6] ends, so it is taken too. Answer: 3.

M20-02 — Merge Overlapping Intervals

Difficulty: Easy | Category: Greedy Algorithms



Sample input

```
4
1 3
2 6
8 10
10 12
```

Sample output

```
1 6
8 12
```

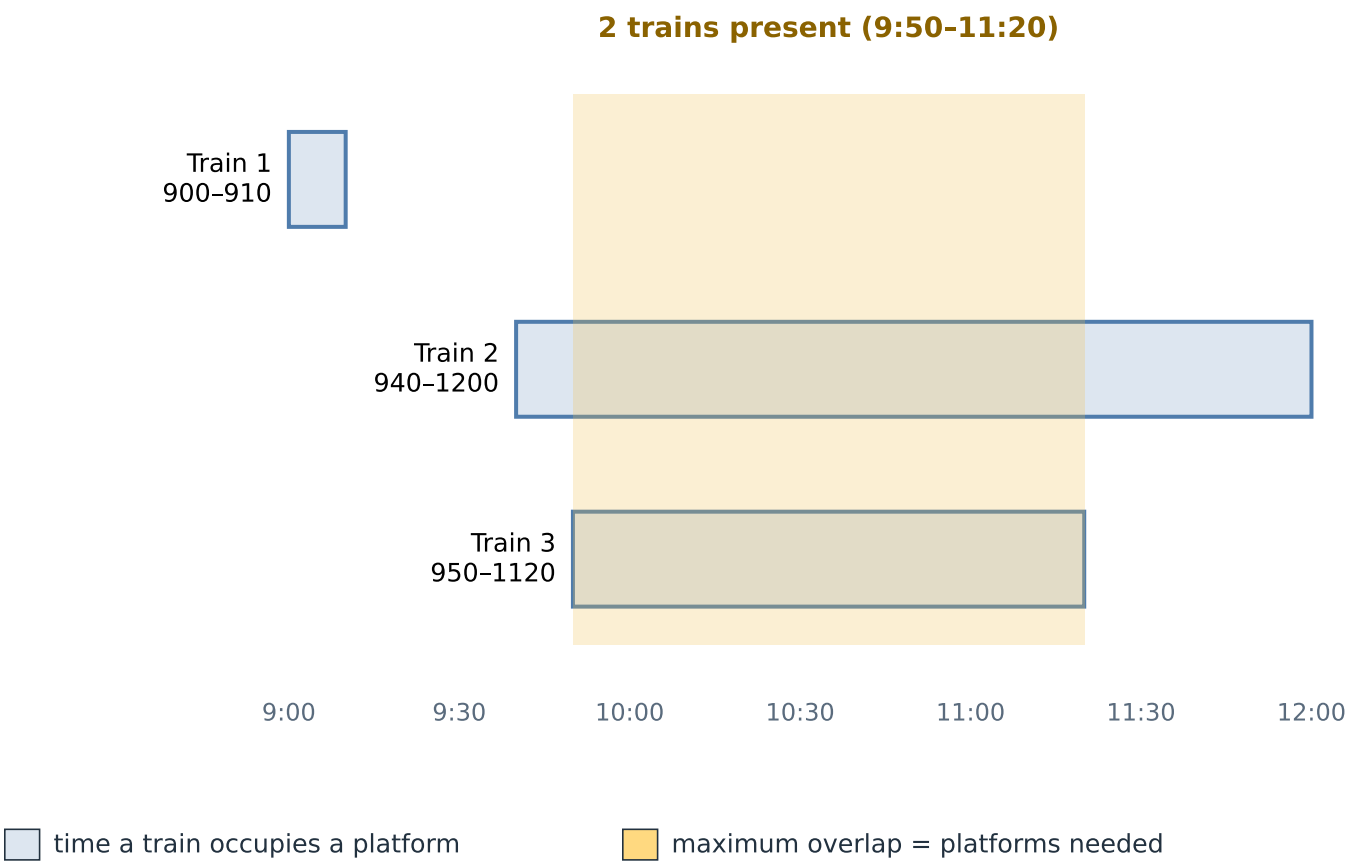
Top: intervals sorted by start. Bottom: the merged result (green).

Walkthrough

[2,6] starts at 2 ≤ 3, so it extends [1,3] to [1,6]. [8,10] starts after 6 and opens a new block; [10,12] starts at 10 ≤ 10 (touching), so the block becomes [8

M20-03 — Minimum Number of Platforms

Difficulty: Easy | Category: Greedy Algorithms



Bars show when each train occupies a platform; the shaded band is where two trains overlap.

Sample input

```
3
900 940 950
910 1200 1120
```

Sample output

```
2
```

Walkthrough

Train 1 has left (9:10) before trains 2 and 3 arrive. Trains 2 and 3 are both present from 9:50 to 11:20, so 2 platforms are needed and never 3.

M20-04 — Fractional Knapsack

Difficulty: Easy | Category: Greedy Algorithms

Item	Weight	Value	Value / weight	Taken
1	10	60	6.00	all 10
2	20	100	5.00	all 20
3	30	120	4.00	20 of 30

Bag capacity W = 50



Total value = 60 + 100 + 80 = 240.00

Sample input

```
3 50
10 60
20 100
30 120
```

Sample output

```
240.00
```

Items sorted by value per unit of weight. Green items are taken whole; the gold item is taken partially.

Walkthrough

Take item 1 (60) and item 2 (100), using 30 of the 50 units. The remaining 20 units take two-thirds of item 3, worth 80. Total 240.00.

M20-05 — Job Sequencing with Deadlines

Difficulty: Medium | Category: Greedy Algorithms

Jobs by profit (high → low)

profit 100, deadline 2 → slot 2

profit 27, deadline 2 → slot 1

profit 25, deadline 1 ✗ no free slot ≤ 1

profit 19, deadline 1 ✗ no free slot ≤ 1

Time slots

slot 1	slot 2
profit 27	profit 100

2 jobs, total profit 127

Sample input

```
4
2 100
1 19
2 27
1 25
```

Sample output

```
2 127
```

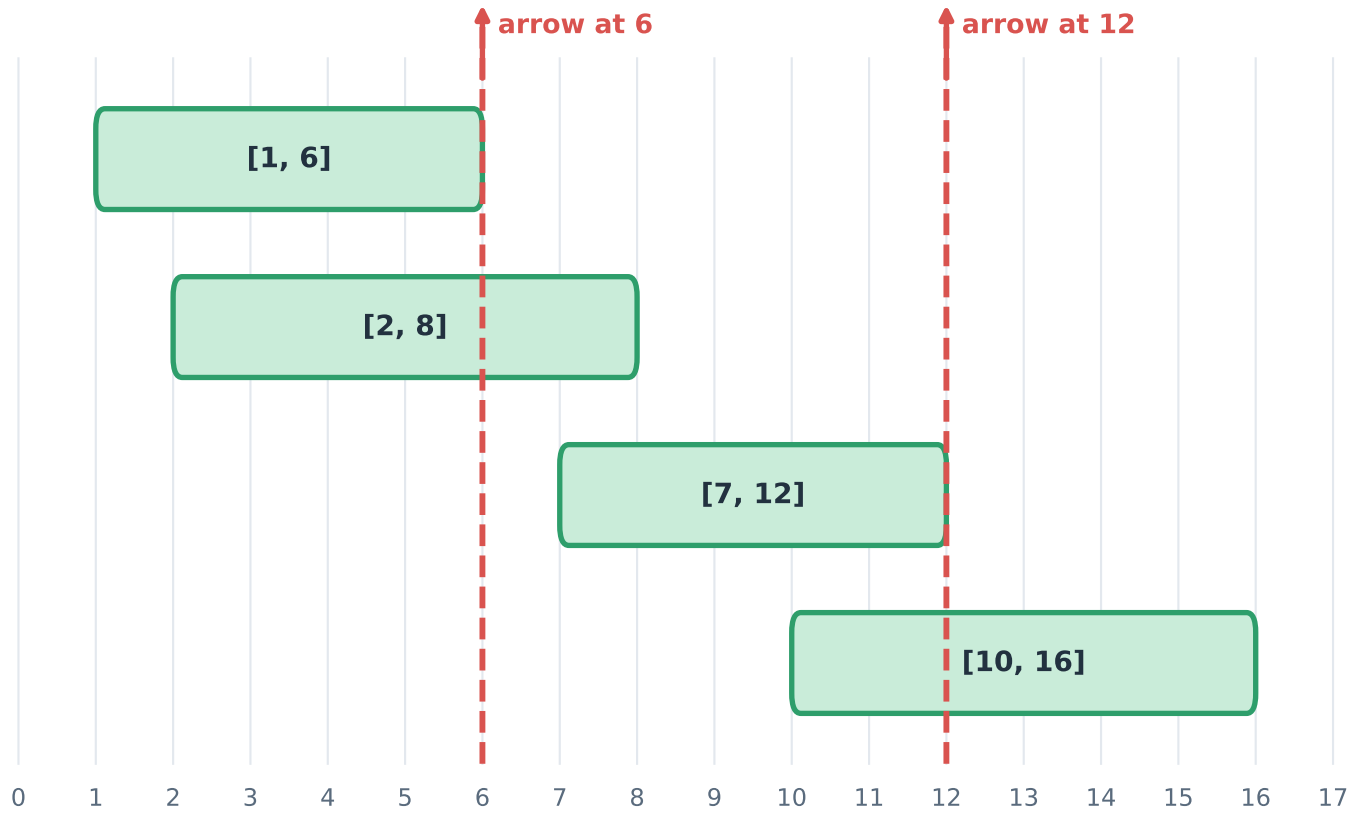
Jobs are tried from highest profit to lowest, each placed in the latest free slot at or before its deadline.

Walkthrough

Profit 100 takes slot 2; profit 27 (deadline 2) takes slot 1. Profits 25 and 19 need slot 1, which is used. Result: 2 jobs, profit 127.

M20-06 — Minimum Arrows to Burst Balloons

Difficulty: Medium | Category: Greedy Algorithms



Sample input

```
4
1 6
2 8
7 12
10 16
```

Sample output

```
2
```

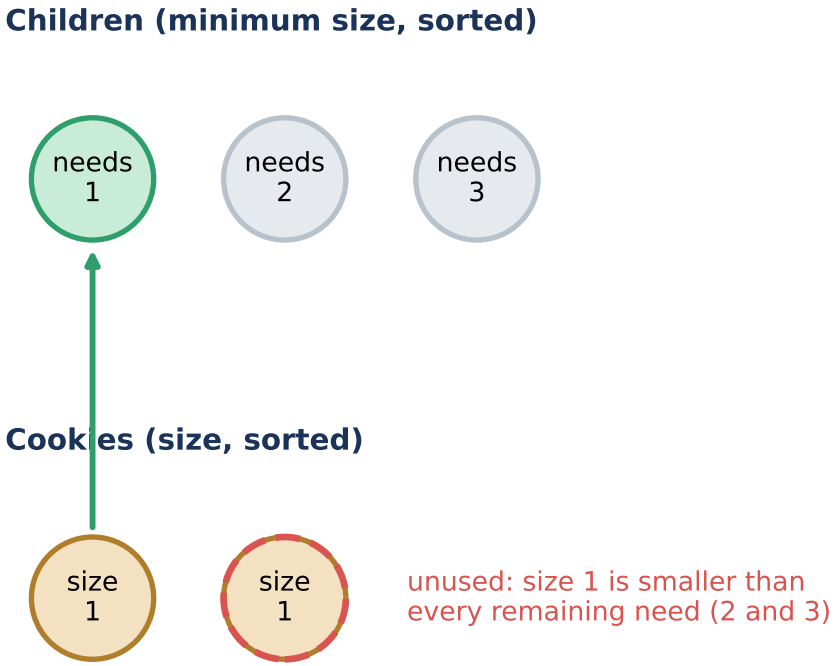
Bars are balloons; red dashed lines are arrows, placed at the smallest right endpoint not yet covered.

Walkthrough

The first arrow goes at 6, the end of [1,6]; it also bursts [2,8] (2 ≤ 6). [7,12] starts after 6, so the second arrow goes at 12, which also bursts [10,16]. Ans 2.

M20-07 — Assign Cookies

Difficulty: Easy | Category: Greedy Algorithms



Sample input

```
3 2
1 2 3
1 1
```

Sample output

```
1
```

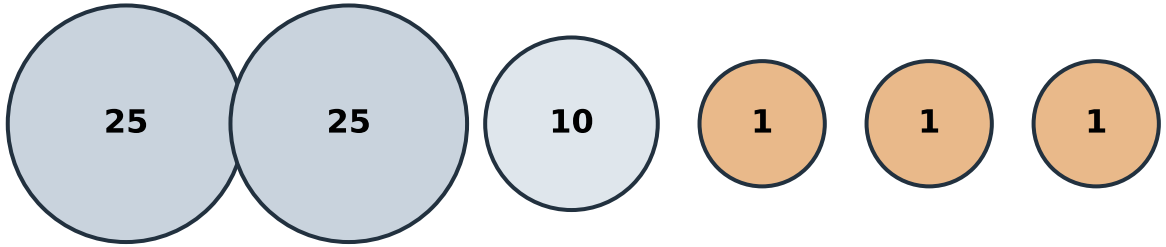
Green: a satisfied child and the cookie given to them.

Walkthrough

The first size-1 cookie satisfies the child who needs 1. The second size-1 cookie is too small for the children needing 2 and 3. Answer: 1.

M20-08 — Coin Change with Fewest Coins (Canonical Coins)

Difficulty: Medium | Category: Greedy Algorithms



63 ÷ 25 = 2 r 13 → 13 ÷ 10 = 1 r 3 → 3 ÷ 5 = 0 r 3 → 3 ÷ 1 = 3

2 + 1 + 0 + 3 = 6 coins

Sample input

63

Sample output

6

The greedy breakdown of 63 using integer division, largest coin first.

Walkthrough

Two 25s leave 13; one 10 leaves 3; no 5 fits; three 1s finish it. Total: 6 coins, which is optimal for the canonical 1/5/10/25 system.

M20-09 — 0/1 Knapsack Maximum Value

Difficulty: Medium | Category: Dynamic Programming

capacity c

items considered (weight, value)	0	1	2	3	4	5	6	7
none	0	0	0	0	0	0	0	0
+(1,1)	0	1	1	1	1	1	1	1
+(3,4)	0	1	1	4	5	5	5	5
+(4,5)	0	1	1	4	5	6	6	9
+(5,7)	0	1	1	4	5	7	8	9

Sample input

```
4 7
1 1
3 4
4 5
5 7
```

Sample output

```
9
```

Each cell is the best value with capacity c using the items in that row and above. Gold: the answer.

Walkthrough

In row +(4,5) at capacity 7, taking item (4,5) adds 5 to the best value at capacity 3 in the row above (4, green): 4 + 5 = 9. Item (5,7) cannot improve on t
the answer (gold) stays 9.

M20-10 — Count Ways to Make an Amount

Difficulty: Medium | Category: Dynamic Programming

amount

denominations allowed	0	1	2	3	4	5
no coins	1	0	0	0	0	0
coin 1	1	1	1	1	1	1
+ coin 2	1	1	2	2	3	3
+ coin 5	1	1	2	2	3	4

Sample input

```
3 5
1 2 5
```

Sample output

```
4
```

Each cell counts combinations for that amount using only the denominations up to that row. Gold: the answer.

Walkthrough

With coin 1 only, every amount has 1 way. Adding coin 2 gives 3 ways to make 5. Adding coin 5 adds ways[0] = 1 more (the single 5), so 3 + 1 = 4: 1+1+1+1+1, 1+1+1+2, 1+2+2, and 5.

M20-11 — Longest Common Subsequence Length

Difficulty: Medium | Category: Dynamic Programming

B = BDCABA

A = ABCBDAB	∅	B	D	C	A	B	A
∅	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

Green cells: matches on one LCS (BCBA); gold: answer

Sample input

```
ABCBDAB
BDCABA
```

Sample output

```
4
```

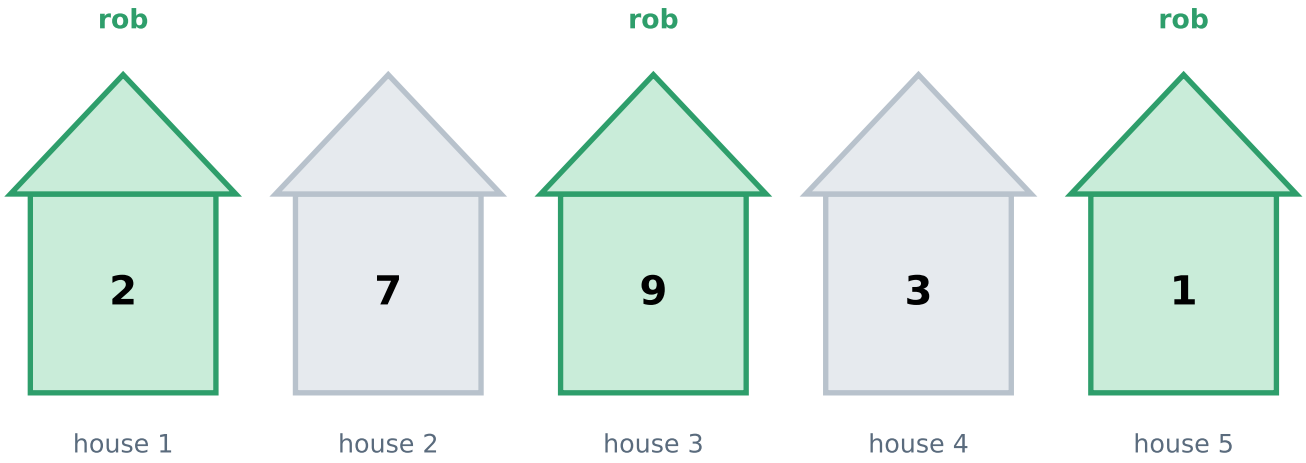
LCS table for ABCBDAB vs BDCABA. Each cell is the LCS length of the two prefixes.

Walkthrough

Matching characters extend the diagonal value by one; otherwise a cell copies the larger of its top and left neighbors. Tracing back the green matches giv
of length 4.

M20-12 — House Robber

Difficulty: Medium | Category: Dynamic Programming



2 + 9 + 1 = 12 (7 + 3 = 10 is worse)

Sample input

```
5
2 7 9 3 1
```

Sample output

```
12
```

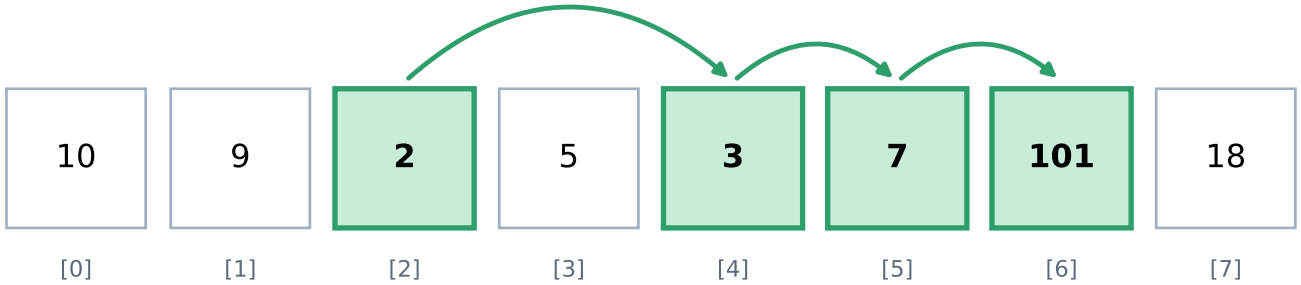
Green houses are robbed; no two robbed houses are adjacent.

Walkthrough

Robbing houses 1, 3, and 5 collects 2 + 9 + 1 = 12. The only other good option, houses 2 and 4, gives 7 + 3 = 10.

M20-13 — Longest Increasing Subsequence Length

Difficulty: Medium | Category: Dynamic Programming



2 < 3 < 7 < 101 → length 4

Sample input

```
8
10 9 2 5 3 7 101 18
```

Sample output

```
4
```

Green cells form one longest strictly increasing subsequence (indexes shown below each value).

Walkthrough

2, 3, 7, 101 keeps the original order and increases strictly, so the length is 4. Other answers of length 4 exist, such as 2, 5, 7, 18.

M20-14 — Edit Distance

Difficulty: Hard | Category: Dynamic Programming

B = sitting

A = kitten

	∅	s	i	t	t	i	n	g
∅	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

Green cells: one optimal alignment path; gold: answer

Sample input

```
kitten
sitting
```

Sample output

```
3
```

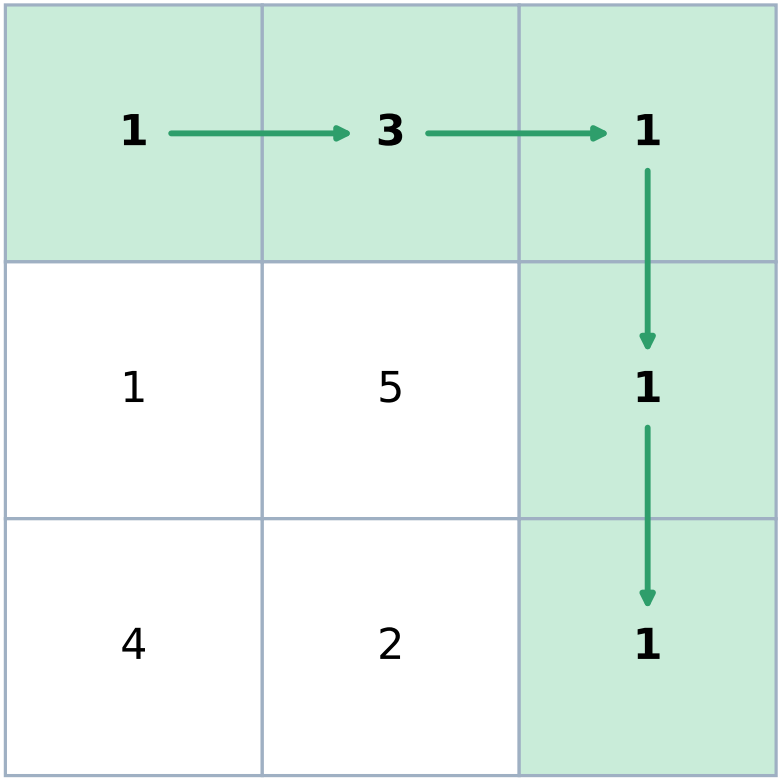
Edit-distance table for kitten → sitting. Each cell is the distance between the two prefixes.

Walkthrough

Following the green path: replace k with s, keep 'itt', replace e with i, keep 'n', and insert g. That is 3 operations.

M20-15 — Minimum Path Sum in a Grid

Difficulty: Medium | Category: Dynamic Programming



Sample input

```
3 3
1 3 1
1 5 1
4 2 1
```

Sample output

```
7
```

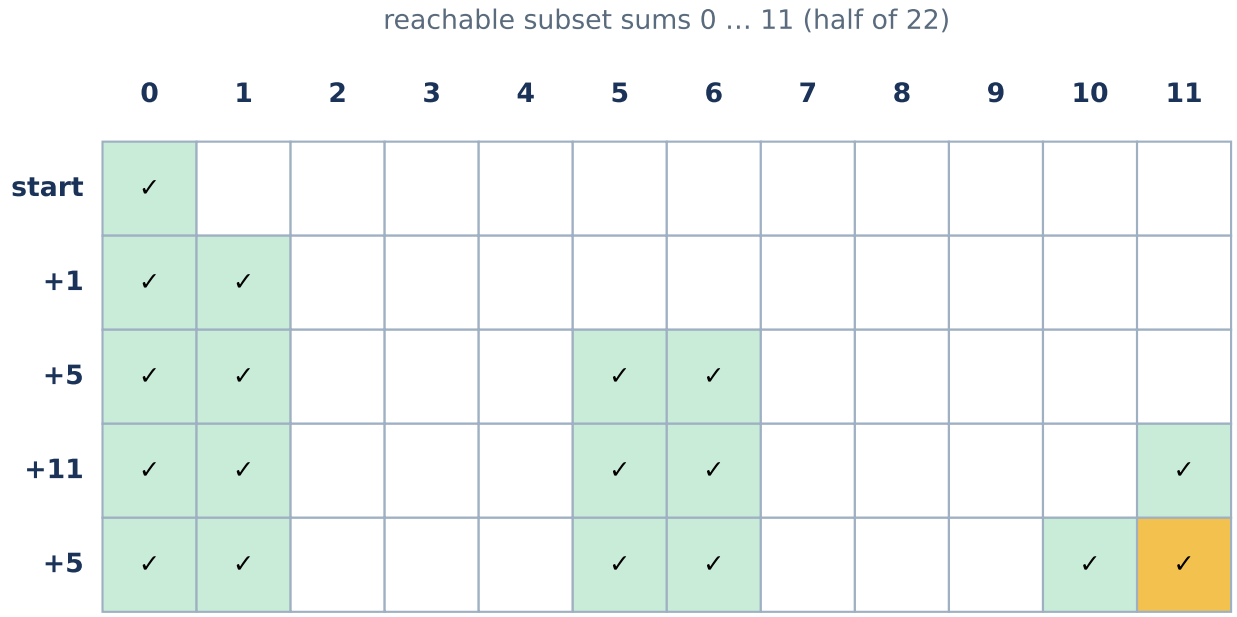
Green cells and arrows show the minimum-sum route (moves only right or down).

Walkthrough

Right, right, down, down visits 1 → 3 → 1 → 1 → 1 for a sum of 7. Any route through the 5 or the 4 costs more.

M20-16 — Partition Equal Subset Sum

Difficulty: Hard | Category: Dynamic Programming



Sample input

```
4
1 5 11 5
```

Sample output

```
YES
```

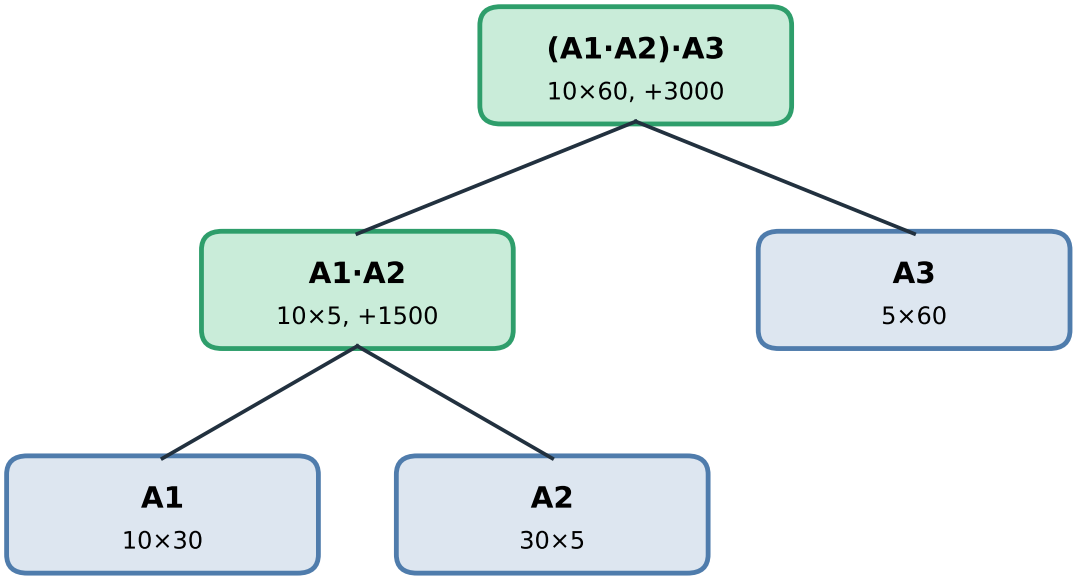
Each row shows which subset sums up to 11 are reachable after adding that element. Gold: the target.

Walkthrough

The total is 22, so the target is 11. After adding 11 as the third element, sum 11 becomes reachable: {11} and {1, 5, 5} each sum to 11. Answer: YES.

M20-17 — Matrix Chain Multiplication

Difficulty: Hard | Category: Dynamic Programming



Best: 1500 + 3000 = 4500

Other order A1·(A2·A3): 30×5×60 + 10×30×60 = 9000 + 18000 = 27000

Sample input

```
3
10 30 5 60
```

Sample output

```
4500
```

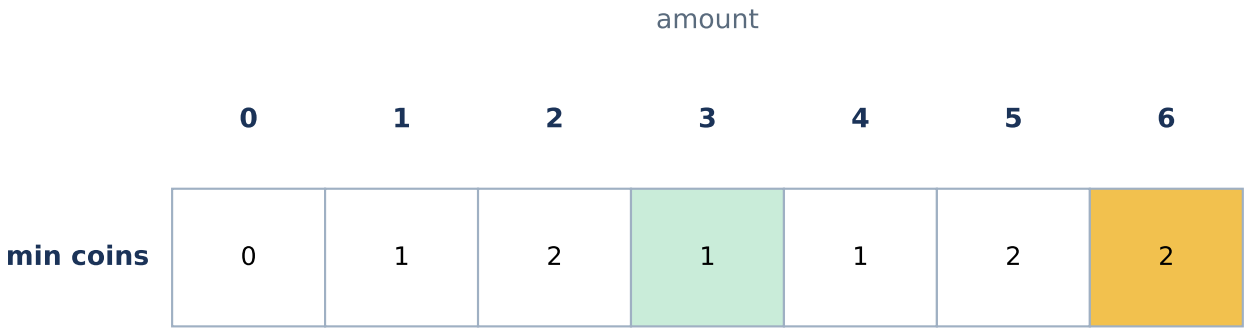
Multiplication tree for the best parenthesization; each node shows the result size and the cost added there.

Walkthrough

Multiplying A1·A2 first produces a small 10×5 matrix, so the final step costs only 10×5×60. Total 4500, compared with 27000 for the other order.

M20-18 — Minimum Coins for Arbitrary Denominations

Difficulty: Hard | Category: Dynamic Programming



Optimal (DP): 3 + 3 → 2 coins

Greedy: 4 + 1 + 1 → 3 coins (not optimal)

`best[6] = 1 + min(best[5], best[3], best[2]) = 1 + 1 = 2`

Sample input

```
3 6
1 3 4
```

Sample output

```
2
```

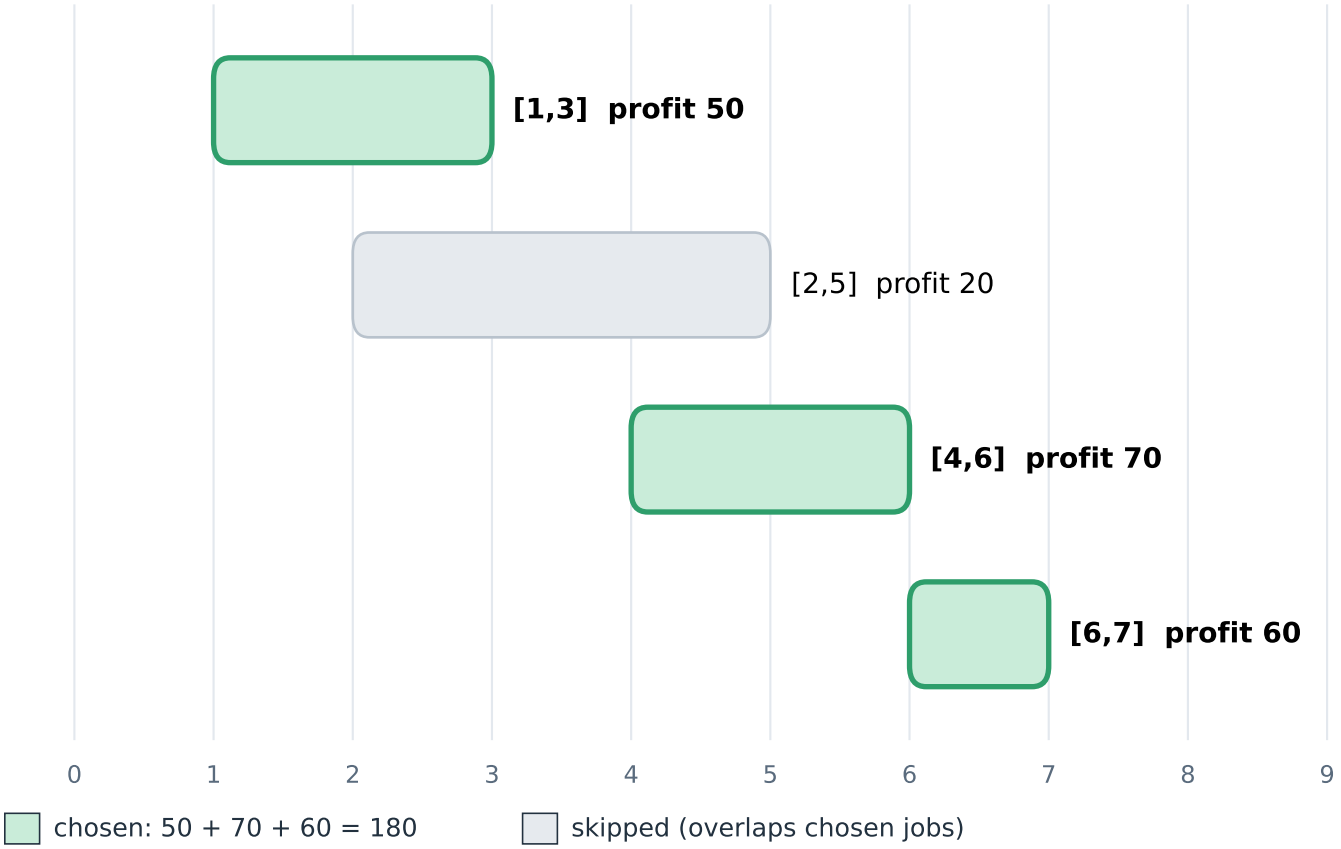
best[x] is the fewest coins that make amount x using coins 1, 3, and 4. Gold: the answer.

Walkthrough

best[6] = 2 (3 + 3). Greedy takes 4 first and then needs two 1s, using 3 coins, so a DP over all amounts is required.

M20-19 — Maximum Profit from Non-overlapping Jobs

Difficulty: Hard | Category: Dynamic Programming



Sample input

```
4
1 3 50
2 5 20
4 6 70
6 7 60
```

Sample output

```
180
```

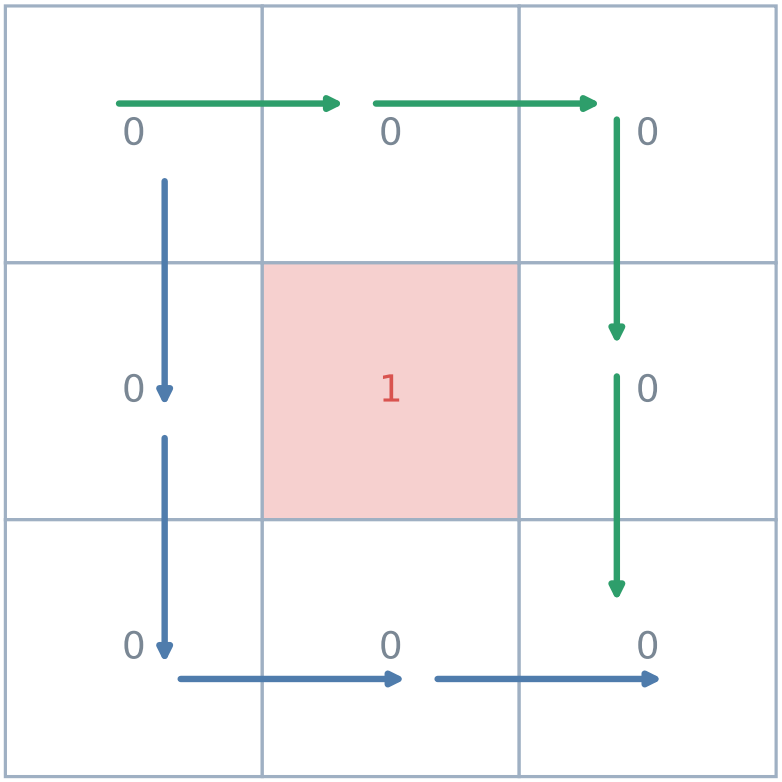
Bars are jobs labelled with their profit; green bars are the chosen non-overlapping set.

Walkthrough

[1,3] (50), [4,6] (70), and [6,7] (60) do not overlap; [6,7] may start exactly when [4,6] ends. [2,5] (20) overlaps both of the first two. Total 180.

M20-20 — Count Paths in a Grid with Obstacles

Difficulty: Medium | Category: Dynamic Programming



Sample input

```
3 3
0 0 0
0 1 0
0 0 0
```

Sample output

```
2
```

The red cell is blocked. Green arrows: route 1 (right, right, down, down). Blue arrows: route 2 (down, down, right, right).

Walkthrough

With the center blocked, a path must go along the top row and down the right column, or down the left column and along the bottom row. Answer: 2.